LAPORAN PRAKTIKUM

POSTTEST 7

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Intan Ayu Rahma Putri (2509106062)

Kelas (B1 ‘25)

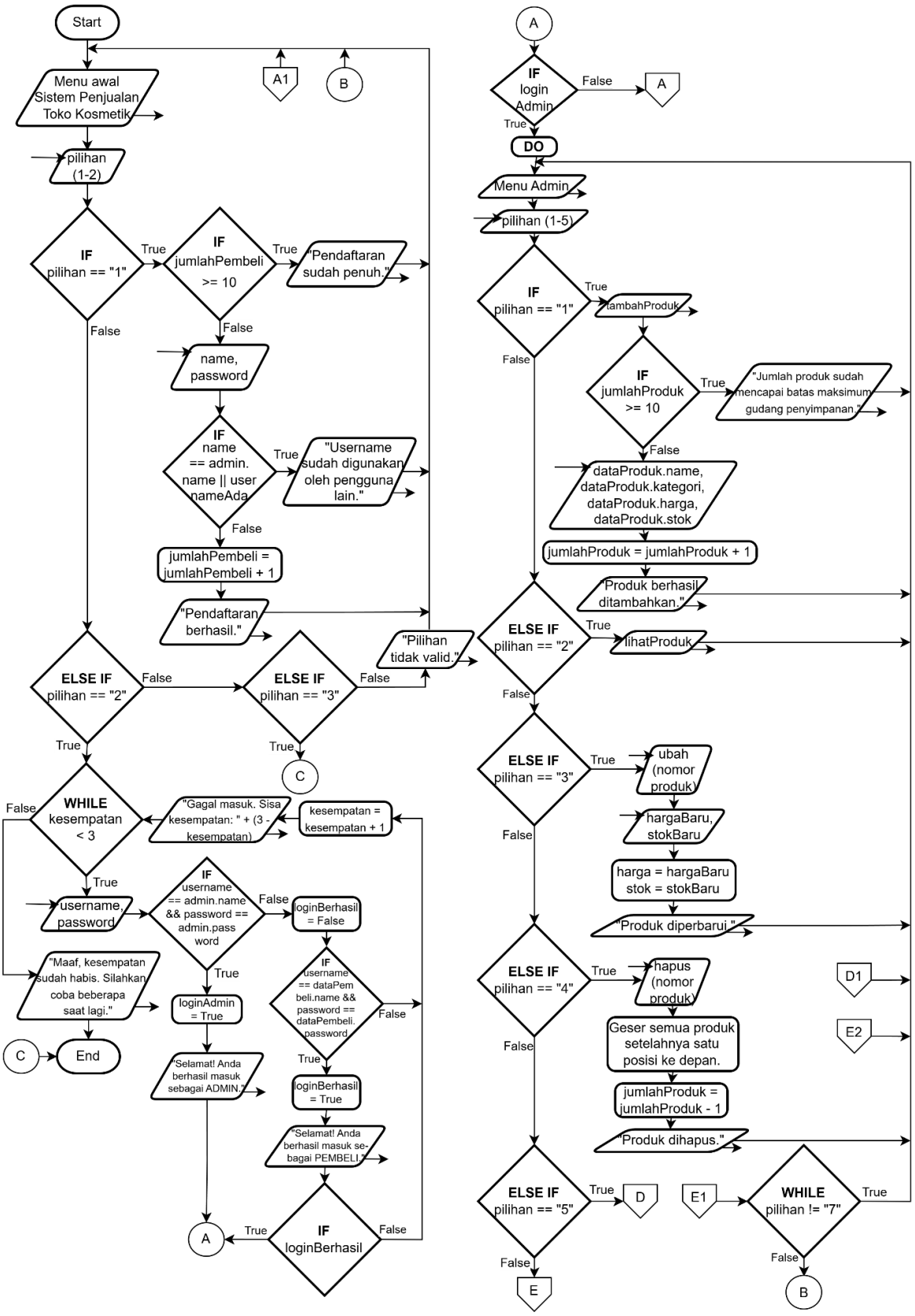
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

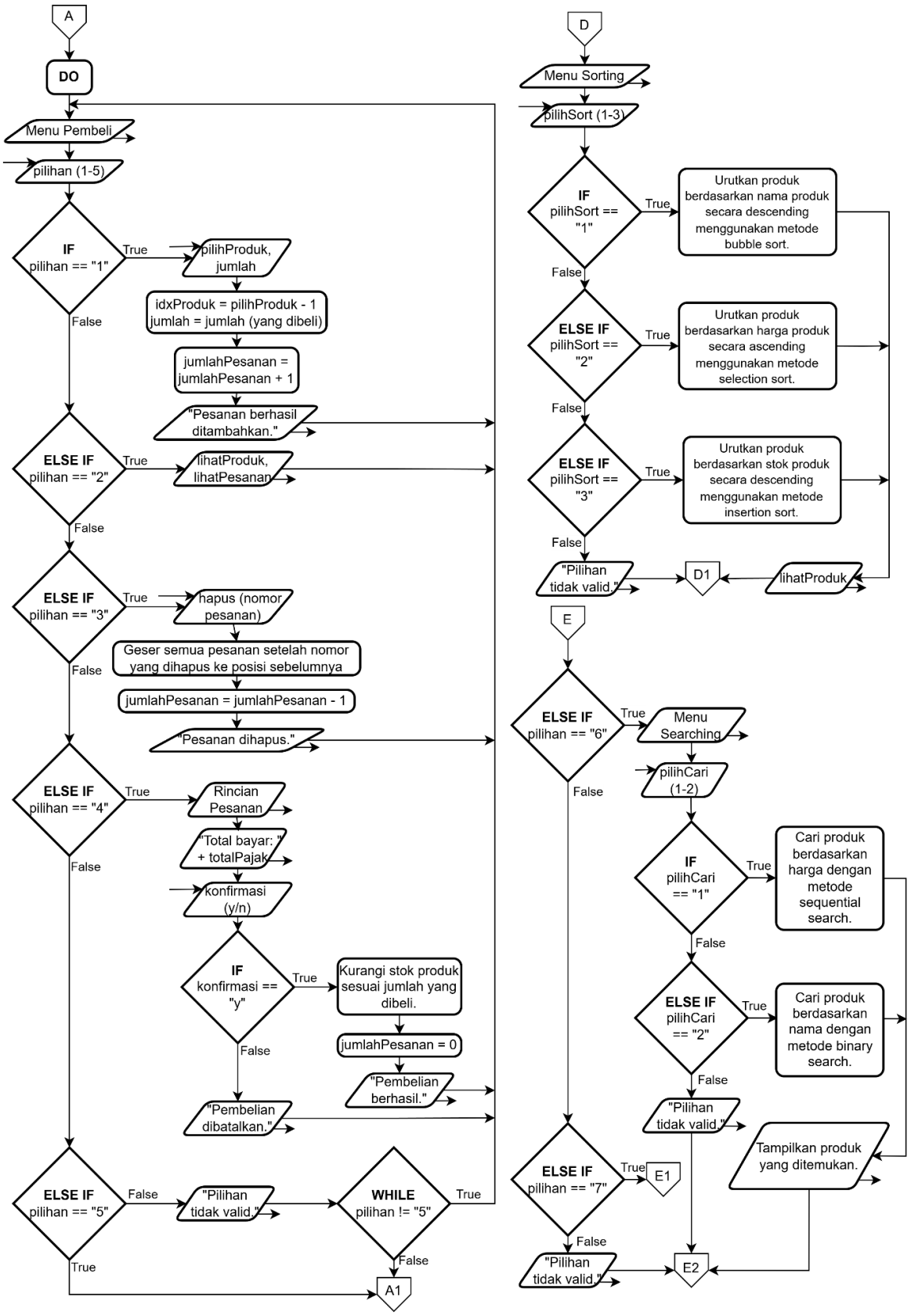
SAMARINDA

2026

1. Flowchart



Gambar 1.1 Flowchart PT 7



Gambar 1.2 Flowchart PT 7

Alur Logika Program

1. Mulai program.

2. Inisialisasi data admin, array dataPembeli, array dataProduk, array dataPesanan, serta variabel jumlahPembeli, jumlahProduk, jumlahPesanan, dan kesempatan.

3. Tampilkan Menu Awal: Sistem Penjualan Toko Kosmetik.

4. Masukkan pilihan utama pengguna (1–3).

5. Proses pilihan utama:

5.1 Jika pilihan == “1” (Daftar):

5.1.1 Periksa apakah jumlahPembeli >= 10.

5.1.2 Jika ya, tampilkan pesan "Pendaftaran sudah penuh".

5.1.3 Jika tidak, Masukkan name dan password pembeli baru.

5.1.4 Periksa apakah name sama dengan admin.name atau sudah ada di data pembeli.

5.1.5 Jika ya, tampilkan "Username sudah digunakan oleh pengguna lain".

5.1.6 Jika tidak, simpan data pembeli baru dan jumlahPembeli = jumlahPembeli + 1.

5.1.7 Tampilkan pesan "Pendaftaran berhasil".

5.2 Jika pilihan == “2”, lanjut ke proses login.

5.3 Jika pilihan == “3”, maka keluar dari Menu Awal dan program selesai.

5.4 Jika pilihan lain, tampilkan "Pilihan tidak valid".

6. Proses login dilakukan dalam perulangan WHILE kesempatan < 3:

6.1 Masukkan username dan password.

6.2 Jika username == admin.name dan password == admin.password:

6.2.1 loginAdmin = True.

6.2.2 loginBerhasil = True.

6.3 Jika tidak, periksa apakah username == dataPembeli.name dan password == dataPembeli.password.

6.3.1 Jika sesuai, loginBerhasil = True.

6.4 Jika tidak sesuai:

6.4.1 loginBerhasil = False.

6.4.2 Tambah kesempatan = kesempatan + 1.

6.4.3 Tampilkan "Gagal masuk. Sisa kesempatan: (3 − kesempatan)".

7. Setelah perulangan login selesai:

7.1 Jika kesempatan == 3, maka program berakhir.

7.2 Jika loginBerhasil == True, lanjut ke penentu peran (Connector A).

8. Menentukan masuk sebagai siapa (admin/pembeli):

8.1 Jika loginAdmin == True maka masuk ke Menu Admin.

8.2 Tampilkan Menu Admin dengan pilihan:

8.2.1 Tambah Produk

8.2.2 Lihat Produk

8.2.3 Ubah Produk

8.2.4 Hapus Produk

8.2.5 Urutkan Produk

8.2.6 Cari Produk

8.2.7 Keluar

8.3 Jalankan perulangan DO-WHILE pilihan != “7”.

8.4 Jika pilihan == “1” (Tambah Produk):

8.4.1 Periksa apakah jumlahProduk >= 10.

8.4.2 Jika ya, tampilkan "Jumlah produk sudah mencapai batas maksimum gudang penyimpanan".

8.4.3 Jika tidak, Masukkan nama produk, kategori, harga, dan stok.

8.4.4 Tambah jumlahProduk = jumlahProduk + 1.

8.4.5 Tampilkan "Produk berhasil ditambahkan".

8.5 Jika pilihan == “2”, tampilkan lihatProduk.

8.6 Jika pilihan == “3” (Ubah Produk):

8.6.1 Masukkan nomor produk yang diubah.

8.6.2 Masukkan hargaBaru dan stokBaru.

8.6.3 Ubah harga = hargaBaru dan stok = stokBaru.

8.6.4 Tampilkan "Produk diperbarui".

8.7 Jika pilihan == “4” (Hapus Produk):

8.7.1 Masukkan nomor produk yang dihapus.

8.7.2 Geser data produk setelahnya.

8.7.3 Ubah jumlahProduk = jumlahProduk − 1.

8.7.4 Tampilkan "Produk dihapus".

8.8 Jika pilihan == “5” (Urutkan Produk):

8.8.1 Tampilkan menu pengurutan.

8.8.2 Input pilihSort.

8.8.3 Jika:

8.8.3.1 PilihSort = “1”, maka urutkan nama (Z–A).

8.8.3.2 PilihSort = “2”, maka urutkan harga (ascending).

8.8.3.3 PilihSort = “3”, maka urutkan stok (descending).

8.8.3.4 Tidak valid, maka tampilkan pesan error, lanjut ke 8.8.5.

8.8.4 Tampilkan data hasil pengurutan.

8.8.5 Selesai/kembali ke menu admin.

8.9 Jika pilihan == “6” (Cari Produk):

8.9.1 Tampilkan menu pencarian.

8.9.2 Input pilihCari.

8.9.3 Jika:

8.9.3.1 pilihCari == “1”, maka produk akan dicari berdasarkan harga menggunakan metode sequential search.

8.9.3.2 pilihCari == “2”, maka produk akan dicari berdasarkan nama menggunakan metode binary search.

8.9.3.3 Tidak valid, maka tampilkan pesan error, lanjut ke 8.9.5.

8.9.4 Tampilkan produk yang ditemukan.

8.9.5 Selesai/kembali ke menu admin.

8.10 Jika pilihan == “7”, keluar dari perulangan Menu Admin.

8.11 Jika pilihan tidak sesuai, tampilkan "Pilihan tidak valid".

9. Jika loginAdmin == False maka masuk ke Menu Pembeli.

10. Tampilkan Menu Pembeli dengan pilihan:

10.1 Buat Pesanan

10.2 Lihat Produk & Pesanan

10.3 Hapus Pesanan

10.4 Bayar Pesanan

10.5 Keluar.

11. Jalankan perulangan DO-WHILE pilihan != "5".

12. Jika pilihan == “1” (Buat Pesanan):

12.1 Masukkan pilihProduk dan jumlah.

12.2 Hitung idxProduk = pilihProduk − 1.

12.3 Tambah jumlahPesanan = jumlahPesanan + 1.

12.4 Tampilkan "Pesanan berhasil ditambahkan".

13. Jika pilihan == “2”, tampilkan lihatProduk dan lihatPesanan.

14. Jika pilihan == “3” (Hapus Pesanan):

14.1 Masukkan nomor pesanan yang dihapus.

14.2 Geser data pesanan setelahnya.

14.3 Ubah jumlahPesanan = jumlahPesanan − 1.

14.4 Tampilkan "Pesanan dihapus".

15. Jika pilihan == “4” (Bayar Pesanan):

15.1 Tampilkan Rincian Pesanan.

15.2 Tampilkan Total bayar: totalPajak.

15.3 Masukkan konfirmasi (y/n).

15.4 Jika konfirmasi == “y”:

15.4.1 Kurangi stok produk sesuai jumlah yang dibeli.

15.4.2 JumlahPesanan = 0.

15.4.3 Tampilkan "Pembelian berhasil".

15.5 Jika tidak, tampilkan "Pembelian dibatalkan".

16. Jika pilihan == “5”, keluar dari perulangan Menu Pembeli.

17. Jika pilihan tidak sesuai, tampilkan "Pilihan tidak valid".

18. Program selesai.

2. Deskripsi Singkat Program

Program “Sistem Penjualan Toko Kosmetik” dibuat dengan tujuan melakukan proses penjualan produk kosmetik pada sebuah toko bernama AYE’S COSMETICS menggunakan bahasa pemrograman C++. Program menyediakan fitur pendaftaran akun pembeli dan masuk sehingga hanya pengguna yang terdaftar yang dapat mengakses sistem, juga mempunyai dua jenis pengguna, yaitu admin dan pembeli. Admin bertugas mengelola data produk seperti menambah, melihat, mengubah, mengurutkan, mencari, dan menghapus produk. Sementara itu, pembeli dapat melihat produk, membuat pesanan, melihat pesanan, menghapus pesanan, serta melakukan pembayaran. Selain itu, baik menu admin maupun menu pembeli memiliki fitur keluar.

3. Source Code

A. Fitur Inisialisasi Data

Di awal program, sistem menyiapkan semua data yang diperlukan, seperti data admin, data pembeli, daftar produk kosmetik, dan tempat untuk menyimpan pesanan.

Source Code:

```cpp
#include <iostream>
#include <stdexcept>
#include <string>
#include <limits>
#include <iomanip>
#include <algorithm>
#include "hitung_toko.h"

using namespace std;
using namespace TokoLib;

#define RESET   "\033[0m"
#define RED     "\033[31m"
#define GREEN   "\033[32m"
#define YELLOW  "\033[33m"
#define CYAN    "\033[36m"

struct Detail {
    int harga;
    int stok;
};

struct Produk {
    string name;
    string kategori;
    Detail detail;
};

struct Pesanan {
    int idxProduk;
    int jumlah;
};

struct Pengguna {
    string name;
    string password;
};

int main() {
    Pengguna admin = { "Aya", "062" };
    Pengguna dataPembeli[10] = {
        {"Ayu", "123"}
    };

    Produk dataProduk[10] = {
        {"AYE! Nude Eyeshadow Pallete", "Eye Makeup", {55000, 10}},
        {"AYE! Coral Glossy Tint", "Lip Makeup", {35000, 10}},
        {"AYE! Autumn Brown Blush", "Cheek Makeup", {40000, 10}},
        {"AYE! Curling Black Mascara", "Eye Makeup", {79000, 10}},
        {"AYE! Ivory Cushion W/ SPF++", "Base Makeup", {98000, 10}}
    };
```

```
    Pesanan dataPesanan[10];

    int jumlahProduk = 5;
    int jumlahPesanan = 0;
    int jumlahPembeli = 1;

    string enter;
    string username, password;
    bool loginAdmin = false;
    bool loginPembeli = false;
    int kesempatan;
    string pilihan;
```

B. Fitur Pendaftaran Pembeli

Fitur ini digunakan jika pengguna belum punya akun. Pengguna bisa membuat akun dengan memasukkan username dan password. Sistem akan mengecek apakah username sudah dipakai atau belum.

Source Code:

```
    while (true) {
        try {
            clearScreen();
            cout << CYAN << "=============================================\n";
            cout << "     SISTEM PENJUALAN TOKO KOSMETIK >.<      \n";
            cout << "=============================================\n" << RESET;
            cout << "1. Daftar\n2. Masuk\n3. Keluar\n\nPilih (1/2/3): ";
getline(cin, pilihan);
            if (pilihan.empty()) throw invalid_argument("Input tidak boleh
kosong!");

            if (pilihan == "1") {
                if (jumlahPembeli >= 10) {
                    cout << "\nPendaftaran sudah penuh.\n";
                    pause();
                } else {
                    clearScreen();
                    cout << "=== PENDAFTARAN PEMBELI U_U ===\n";
                    try {
                        cout << "Username: "; getline(cin, username);
                        if (username.empty()) {
                            throw invalid_argument("Username tidak boleh
kosong.");
```

```
                        }

                        cout << "Password: "; getline(cin, password);
                        if (password.empty()) {
                            throw invalid_argument("Password tidak boleh
kosong.");
                        }

                        string inputName = username;
                        string inputPass = password;

                        bool usernameAda = false;
                        for (int i = 0; i < jumlahPembeli; i++) {
                            if (inputName == dataPembeli[i].name) {
                                usernameAda = true;
                                break;
                            }
                        }

                        if (inputName == admin.name || usernameAda) {
                            throw logic_error("Username sudah digunakan.");
                        }

                        dataPembeli[jumlahPembeli].name = inputName;
                        dataPembeli[jumlahPembeli].password = inputPass;
                        jumlahPembeli++;
                        cout << GREEN << "Pendaftaran berhasil.\n" << RESET;
                    } catch (const exception& e) {
                        cout << RED << "\n[ERROR] " << e.what() << RESET <<
endl;
                    }
                }
                pause();
```

C. Fitur Masuk

Fitur ini digunakan jika pengguna sudah punya akun. Pengguna bisa masuk dengan memasukkan username dan password. Sistem akan mengecek apakah username sudah benar atau belum.

Source Code:

```
            } else if (pilihan == "2") {
                loginAdmin = false;
                loginPembeli = false;
```

```
                kesempatan = 0;
                while (kesempatan < 3) {
                    clearScreen();
                    cout << "\n=== MASUK KE AKUN O,O ===\n";
                    try {
                        cout << "Username: "; getline(cin, username);
                        if (username.empty()) {
                            throw invalid_argument("Username tidak boleh
kosong.");
                        }

                        cout << "Password: "; getline(cin, password);
                        if (password.empty()) {
                            throw invalid_argument("Password tidak boleh
kosong.");
                        }

                        if (username == admin.name && password ==
admin.password) {
                            loginAdmin = true;
                            cout << GREEN << "\nSelamat! Anda berhasil masuk
sebagai ADMIN.\n" << RESET;
                            pause();
                            break;
                        }

                        bool loginBerhasil = false;
                        for (int i = 0; i < jumlahPembeli; i++) {
                            if (username == dataPembeli[i].name && password ==
dataPembeli[i].password) {
                                loginBerhasil = true;
                                loginPembeli = true;
                                jumlahPesanan = 0;
                                cout << GREEN << "\nSelamat! Anda berhasil masuk
sebagai PEMBELI.\n" << RESET;
                                pause();
                                break;
                            }
                        }

                        if (!loginBerhasil) {
                            throw runtime_error("Username atau password
salah!");
                        } else {
                            kesempatan = 0;
                            break;
                        }
                    } catch (const exception& e) {
                        kesempatan++;
                        cout << RED << "\n[ERROR] " << e.what() << RESET <<
endl;
```

```
                        cout << "\nSisa kesempatan: " << 3 - kesempatan << endl;

                        if (kesempatan < 3) {
                            cout << YELLOW << "Silakan coba lagi.\n" << RESET;
                            pause();
                        }

                        if (kesempatan == 3) {
                            cout << RED << "Kesempatan login habis! Silakan coba
beberapa saat lagi." << RESET;
                            return 0;
                        }
                    }
                }
```

D. Fitur Menu Admin

Jika yang masuk adalah admin, maka admin bisa mengelola produk di toko. Admin bisa menambah produk baru, melihat daftar produk, mengubah harga atau stok produk, mengurutkan produk, mencari produk, dan juga menghapus produk.

Source Code:

```
                if (loginAdmin) {
                    do {
                        clearScreen();
                        cout << "\n=== MENU ADMIN >u< ===\n";
                        cout << "1. Tambah Produk\n";
                        cout << "2. Lihat Produk\n";
                        cout << "3. Ubah Produk\n";
                        cout << "4. Hapus Produk\n";
                        cout << "5. Urutkan Produk\n";
                        cout << "6. Cari Produk\n";
                        cout << "7. Keluar\n";
                        cout << "---------------------\nPilih: "; getline(cin,
pilihan);
                        if (pilihan.empty()) throw invalid_argument("Input tidak
boleh kosong!");

                        if (pilihan == "1") {
                            clearScreen();
                            tambahProduk(jumlahProduk, dataProduk);
                            pause();
                        } else if (pilihan == "2") {
                            clearScreen();
```

```
                            lihatProduk(dataProduk, jumlahProduk);
                            pause();
                        } else if (pilihan == "3") {
                            clearScreen();
                            lihatProduk(dataProduk, jumlahProduk);
                            ubahProduk(jumlahProduk, dataProduk);
                            pause();
                        } else if (pilihan == "4") {
                            clearScreen();
                            lihatProduk(dataProduk, jumlahProduk);
                            hapusProduk(jumlahProduk, dataProduk);
                            pause();
                        } else if (pilihan == "5") {
                            clearScreen();
                            string pilihSort;
                            cout << "\n======== MENU PENGURUTAN PRODUK >;<
========\n";
                            cout << "1. Sort Berdasarkan Nama (Z-A)\n";
                            cout << "2. Sort Berdasarkan Harga (Murah-Mahal)\n";
                            cout << "3. Sort Berdasarkan Stok (Terbanyak)\n";
                            cout <<
"--------------------------------------------\n";
                            cout << "Pilih: ";
                            getline(cin, pilihSort);
                            if (pilihSort.empty()) throw invalid_argument("Input
tidak boleh kosong!");

                            if (pilihSort == "1") {
                                sortNamaDesc(dataProduk, jumlahProduk);
                                lihatProduk(dataProduk, jumlahProduk);
                            } else if (pilihSort == "2") {
                                sortHargaAsc(dataProduk, jumlahProduk);
                                lihatProduk(dataProduk, jumlahProduk);
                            } else if (pilihSort == "3") {
                                sortStokDesc(dataProduk, jumlahProduk);
                                lihatProduk(dataProduk, jumlahProduk);
                            } else {
                                cout << RED << "Pilihan tidak valid.\n" <<
RESET;
                            }
                            pause();
                        } else if (pilihan == "6") {
                            clearScreen();
                            try {
                                string pilihCari;
                                cout << "\n===== MENU SEARCHING =====\n";
                                cout << "1. Cari berdasarkan Harga\n";
//sequential search
                                cout << "2. Cari berdasarkan Nama\n"; //binary
search
                                cout << "--------------------------\n";
```

```
                                cout << "Pilih: "; getline(cin, pilihCari);
                                if (pilihCari.empty()) throw
invalid_argument("Input tidak boleh kosong!");

                                if (pilihCari == "1") {
                                    string tempHarga; cout << "\nMasukkan harga
yang dicari: "; getline(cin, tempHarga);
                                    if (tempHarga.empty()) throw
invalid_argument("Input tidak boleh kosong!");
                                    if (!all_of(tempHarga.begin(),
tempHarga.end(), ::isdigit)) throw invalid_argument("Input harga harus angka!");
                                    int harga = stoi(tempHarga);
                                    int hasil =
sequentialSearchHarga(dataProduk, jumlahProduk, harga);

                                    if (hasil != -1) {
                                        cout << "\nProduk ditemukan:\n";
                                        cout << dataProduk[hasil].name << " | "
<< dataProduk[hasil].kategori << " | Rp" << dataProduk[hasil].detail.harga <<
endl;
                                    } else {
                                        cout << "[!] Produk tidak ditemukan.\n";
                                    }
                                } else if (pilihCari == "2") {
                                    string nama; cout << "\nMasukkan nama
produk: "; getline(cin, nama);
                                    if (nama.empty()) {
                                        throw invalid_argument("Nama produk
tidak boleh kosong!");
                                    }

                                    sortNamaAsc(dataProduk, jumlahProduk);
                                    int hasil = binarySearchNama(dataProduk,
jumlahProduk, nama);

                                    if (hasil != -1) {
                                        cout << "\nProduk ditemukan:\n";
                                        cout << dataProduk[hasil].name << " | "
<< dataProduk[hasil].kategori << " | Rp" << dataProduk[hasil].detail.harga <<
endl;
                                    } else {
                                        cout << "[!] Produk tidak ditemukan.\n";
                                    }
                                } else {
                                    cout << RED << "Pilihan tidak valid.\n" <<
RESET;
                                }
                            } catch (const exception& e) {
                                cout << RED << "\n[ERROR] " << e.what() << RESET
<< endl;
                            }
```

```
                            pause();
                        } else if (pilihan == "7") {
                            break;
                        } else {
                            cout << RED << "\nPilihan tidak valid." << RESET <<
endl;
                            pause();
                        }
                    } while (pilihan != "7");
```

E. Fitur Menu Pembeli

Jika yang login adalah pembeli, maka pembeli bisa melihat produk, membuat pesanan, melihat pesanan yang sudah dibuat, menghapus pesanan, dan melakukan pembayaran.

Source Code:

```
                } else if (loginPembeli) {
                    do {
                        clearScreen();
                        cout << "\n==== MENU PEMBELI ^w^ ====\n";
                        cout << "1. Buat Pesanan\n";
                        cout << "2. Lihat Produk & Pesanan\n";
                        cout << "3. Hapus Pesanan\n";
                        cout << "4. Bayar Pesanan\n";
                        cout << "5. Keluar\n";
                        cout << "-------------------------\nPilih: ";
getline(cin, pilihan);
                        if (pilihan.empty()) throw invalid_argument("Input tidak
boleh kosong!");

                        if (pilihan == "1") {
                            clearScreen();
                            lihatProduk(dataProduk, jumlahProduk);
                            buatPesanan(jumlahProduk, dataProduk, dataPesanan,
jumlahPesanan);
                            pause();
                        } else if (pilihan == "2") {
                            clearScreen();
                            lihatProduk(dataProduk, jumlahProduk);
                            lihatPesanan(dataPesanan, dataProduk,
jumlahPesanan);
                            pause();
                        } else if (pilihan == "3") {
                            clearScreen();
                            lihatPesanan(dataPesanan, dataProduk,
```

```
jumlahPesanan);
                            hapusPesanan(dataPesanan, jumlahPesanan);
                            pause();
                        } else if (pilihan == "4") {
                            clearScreen();
                            bayarPesanan(dataPesanan, dataProduk,
jumlahPesanan);
                            pause();
                        } else if (pilihan == "5") {
                            break;
                        } else {
                            cout << RED << "\nPilihan tidak valid." << RESET <<
endl;
                            pause();
                        }
                    } while (pilihan != "5");
                }
```

F. Fitur Keluar

Jika pengguna selesai menggunakan sistem, program akan menampilkan pesan terima kasih lalu program selesai.

Source Code:

```
            } else if (pilihan == "3") {
                clearScreen();
                cout << GREEN << "\nTerima kasih telah mengunjungi AYE'S
COSMETICS.\n" << RESET;
                break;
            } else {
                cout << RED << "\nPilihan tidak valid.\n" << RESET;
                pause();
            }
        } catch (const exception& e) {
            cout << RED << "\n[ERROR] Terjadi Kesalahan: " << e.what() << RESET
<< endl;
            pause();
        }
    }
    return 0;
}
```

G. Fitur Manajemen Produk untuk Admin

Admin dapat melakukan operasi CRUD terhadap data produk yaitu menambah produk baru, melihat daftar produk, mengubah harga dan stok produk yang sudah ada, mengurutkan produk, mencari produk, serta menghapus produk.

Source Code:

```
void lihatProduk(Produk dataProduk[], int jumlahProduk) {
    cout << CYAN <<
"==========================================================================\n";
    cout << left << setw(4) << "No" << setw(32) << "Nama Produk" << setw(18) <<
"Kategori" << setw(12) << "Harga" << "Stok\n";
    cout <<
"==========================================================================\n" <<
RESET;

    for (int i = 0; i < jumlahProduk; i++) {
        cout << left << setw(4) << (i + 1) << setw(32) << dataProduk[i].name <<
setw(18) << dataProduk[i].kategori << "Rp" << setw(10) <<
dataProduk[i].detail.harga << dataProduk[i].detail.stok << endl;
    }
}

void tambahProduk(int &jumlahProduk, Produk dataProduk[]) {
    try {
        cout << "=== TAMBAH PRODUK (z'v'z) ===\n";
        if (jumlahProduk >= 10) {
            throw runtime_error("Gudang Penuh! Maksimal 10 produk.");
        }

        cout << "Nama Produk: "; getline(cin, dataProduk[jumlahProduk].name);
        if (dataProduk[jumlahProduk].name.empty()) {
            throw invalid_argument("Nama produk tidak boleh kosong.");
        }

        cout << "Kategori: "; getline(cin, dataProduk[jumlahProduk].kategori);
        if (dataProduk[jumlahProduk].kategori.empty()) {
            throw invalid_argument("Kategori tidak boleh kosong.");
        }

        cout << "Harga: "; string tempHarga; getline(cin, tempHarga);
        if (tempHarga.empty()) throw invalid_argument("Harga tidak boleh
kosong.");
        if (!all_of(tempHarga.begin(), tempHarga.end(), ::isdigit)) throw
invalid_argument("Harga harus berupa angka.");
        dataProduk[jumlahProduk].detail.harga = stoi(tempHarga);
```

```
        if (!validasiHarga(dataProduk[jumlahProduk].detail.harga)) {
            throw logic_error("Harga minimal adalah Rp20.000.");
        }

        cout << "Stok: "; string tempStok; getline(cin, tempStok);
        if (tempStok.empty()) throw invalid_argument("Stok tidak boleh
kosong.");
        if (!all_of(tempStok.begin(), tempStok.end(), ::isdigit)) throw
invalid_argument("Stok harus berupa angka.");
        dataProduk[jumlahProduk].detail.stok = stoi(tempStok);

        if (dataProduk[jumlahProduk].detail.stok < 0) throw logic_error("Stok
tidak boleh negatif.");

        jumlahProduk++;
        cout << GREEN << "\n[SUCCESS] Produk berhasil ditambahkan.\n" << RESET;
    } catch (const exception& e) {
        cout << RED << "\n[ERROR] Terjadi Kesalahan: " << e.what() << RESET <<
endl;
    }
}

void ubahProduk(int &jumlahProduk, Produk dataProduk[]) {
    try {
        string tempUbah;
        cout << "\nNomor produk: "; getline(cin, tempUbah);
        if (tempUbah.empty()) throw invalid_argument("Nomor produk tidak boleh
kosong.");
        if (!all_of(tempUbah.begin(), tempUbah.end(), ::isdigit)) throw
invalid_argument("Nomor produk harus berupa angka.");
        int ubah = stoi(tempUbah);

        if (ubah < 1 || ubah > jumlahProduk) throw out_of_range("Nomor produk
tidak ditemukan.");

        cout << "Harga baru: "; string tempHarga; getline(cin, tempHarga);
        if (tempHarga.empty()) throw invalid_argument("Harga tidak boleh
kosong.");
        if (!all_of(tempHarga.begin(), tempHarga.end(), ::isdigit)) throw
invalid_argument("Harga harus berupa angka.");
        int hargaBaru = stoi(tempHarga);

        if (!validasiHarga(hargaBaru)) throw logic_error("Harga minimal adalah
Rp20.000.");

        cout << "Stok baru: "; string tempStok; getline(cin, tempStok);
        if (tempStok.empty()) throw invalid_argument("Stok tidak boleh
kosong.");
        if (!all_of(tempStok.begin(), tempStok.end(), ::isdigit)) throw
invalid_argument("Stok harus berupa angka.");
        int stokBaru = stoi(tempStok);
```

```
        if (stokBaru < 0) throw logic_error("Stok tidak boleh negatif.");

        dataProduk[ubah-1].detail.harga = hargaBaru;
        dataProduk[ubah-1].detail.stok = stokBaru;
        cout << GREEN << "\n[SUCCESS] Produk berhasil diperbarui.\n" << RESET;
    } catch (const exception& e) {
        cout << RED << "\n[ERROR] Terjadi Kesalahan: " << e.what() << RESET <<
endl;
    }
}

void hapusProduk(int &jumlahProduk, Produk dataProduk[]) {
    try {
        string tempHapus; cout << "\nNomor produk: "; getline(cin, tempHapus);
        if (tempHapus.empty()) throw invalid_argument("Nomor produk tidak boleh
kosong.");
        if (!all_of(tempHapus.begin(), tempHapus.end(), ::isdigit)) throw
invalid_argument("Nomor produk harus berupa angka.");
        int hapus = stoi(tempHapus);

        if (hapus > 0 && hapus <= jumlahProduk) {
            for (int i = hapus-1; i < jumlahProduk-1; i++) {
                dataProduk[i] = dataProduk[i+1];
            }
            jumlahProduk--;
            cout << GREEN << "\n[SUCCESS] Produk berhasil dihapus.\n" << RESET;
        } else {
            throw out_of_range("Nomor produk tidak ditemukan.");
        }
    } catch (const exception& e) {
        cout << RED << "\n[ERROR] Terjadi Kesalahan: " << e.what() << RESET <<
endl;
    }
}

void sortNamaDesc(Produk dataProduk[], int jumlahProduk) { //bubble sort
    for (int i = 0; i < jumlahProduk - 1; i++) {
        for (int j = 0; j < jumlahProduk - i - 1; j++) {
            if (dataProduk[j].name < dataProduk[j + 1].name) {
                swap(dataProduk[j], dataProduk[j + 1]);
            }
        }
    }
    cout << GREEN << "Produk berhasil diurutkan (Nama Z-A).\n" << RESET;
}

void sortHargaAsc(Produk dataProduk[], int jumlahProduk) { //selection sort
    for (int i = 0; i < jumlahProduk - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < jumlahProduk; j++) {
```

```
            if (dataProduk[j].detail.harga < dataProduk[minIndex].detail.harga)
{
                minIndex = j;
            }
        }
        swap(dataProduk[i], dataProduk[minIndex]);
    }
    cout << GREEN << "Produk berhasil diurutkan (Harga termurah).\n" << RESET;
}

void sortStokDesc(Produk dataProduk[], int jumlahProduk) { //insertion sort
    for (int i = 1; i < jumlahProduk; i++) {
        Produk key = dataProduk[i];
        int j = i - 1;

        while (j >= 0 && dataProduk[j].detail.stok < key.detail.stok) {
            dataProduk[j + 1] = dataProduk[j];
            j--;
        }
        dataProduk[j + 1] = key;
    }
    cout << GREEN << "Produk berhasil diurutkan (Stok terbanyak).\n" << RESET;
}

int sequentialSearchHarga(Produk dataProduk[], int jumlahProduk, int
targetHarga) {
    for (int i = 0; i < jumlahProduk; i++) {
        if (dataProduk[i].detail.harga == targetHarga) {
            return i;
        }
    }
    return -1;
}

void sortNamaAsc(Produk dataProduk[], int jumlahProduk) {
    for (int i = 0; i < jumlahProduk - 1; i++) {
        for (int j = 0; j < jumlahProduk - i - 1; j++) {
            if (dataProduk[j].name > dataProduk[j + 1].name) {
                swap(dataProduk[j], dataProduk[j + 1]);
            }
        }
    }
    cout << GREEN << "Produk berhasil diurutkan (Nama A-Z).\n" << RESET;
}

int binarySearchNama(Produk dataProduk[], int jumlahProduk, string targetNama) {
    int kiri = 0, kanan = jumlahProduk - 1;
    while (kiri <= kanan) {
        int tengah = (kiri + kanan) / 2;

        if (dataProduk[tengah].name == targetNama) {
```

```
            return tengah;
        } else if (dataProduk[tengah].name < targetNama) {
            kiri = tengah + 1;
        } else {
            kanan = tengah - 1;
        }
    }
    return -1;
}
```

H. Fitur Manajemen Pesanan untuk Pembeli

Pembeli dapat membuat pesanan dengan memilih produk dan jumlah yang diinginkan, melihat daftar produk bersamaan dengan rincian pesanan, membuat pesanan, menghapus pesanan, serta melakukan pembayaran.

Source Code:

```
void lihatProduk(Produk dataProduk[], int jumlahProduk) {
    cout << CYAN <<
"==========================================================================\n";
    cout << left << setw(4) << "No" << setw(32) << "Nama Produk" << setw(18) <<
"Kategori" << setw(12) << "Harga" << "Stok\n";
    cout <<
"==========================================================================\n" <<
RESET;

    for (int i = 0; i < jumlahProduk; i++) {
        cout << left << setw(4) << (i + 1) << setw(32) << dataProduk[i].name <<
setw(18) << dataProduk[i].kategori << "Rp" << setw(10) <<
dataProduk[i].detail.harga << dataProduk[i].detail.stok << endl;
    }
}

void lihatPesanan(Pesanan dataPesanan[], Produk dataProduk[], int jumlahPesanan)
{
    cout << "\n==== PESANAN ANDA ====\n";
    if (jumlahPesanan == 0) {
        cout << "Belum ada pesanan.\n";
    } else {
        for (int i = 0; i < jumlahPesanan; i++) {
            int idx = dataPesanan[i].idxProduk;
            cout << i + 1 << ". " << dataProduk[idx].name << " | Jumlah: " <<
dataPesanan[i].jumlah << endl;
        }
```

```
    }
}

void buatPesanan(int jumlahProduk, Produk dataProduk[], Pesanan dataPesanan[],
int &jumlahPesanan) {
    try {
        string tempPilih, tempJumlah; cout << "\nPilih nomor produk: ";
getline(cin, tempPilih);
        if (tempPilih.empty()) throw invalid_argument("Nomor produk tidak boleh
kosong.");
        if (!all_of(tempPilih.begin(), tempPilih.end(), ::isdigit)) throw
invalid_argument("Nomor produk harus berupa angka.");
        int pilihProduk = stoi(tempPilih);

        if (pilihProduk > 0 && pilihProduk <= jumlahProduk) {
            cout << "Jumlah beli: "; getline(cin, tempJumlah);
            if (tempJumlah.empty()) throw invalid_argument("Jumlah beli tidak
boleh kosong.");
            if (!all_of(tempJumlah.begin(), tempJumlah.end(), ::isdigit)) throw
invalid_argument("Jumlah beli harus berupa angka.");
            int jumlah = stoi(tempJumlah);

            if (jumlah <= 0) {
                throw logic_error("Jumlah beli harus lebih dari 0.");
            } else if (jumlahPesanan >= 10) {
                throw runtime_error("Pesanan sudah mencapai batas maksimum.");
            } else if (jumlah <= dataProduk[pilihProduk-1].detail.stok) {
                dataPesanan[jumlahPesanan].idxProduk = pilihProduk-1;
                dataPesanan[jumlahPesanan].jumlah = jumlah;
                jumlahPesanan++;
                cout << GREEN << "\n[SUCCESS] Pesanan berhasil ditambahkan.\n"
<< RESET;
            } else {
                throw runtime_error("Stok tidak cukup.");
            }
        } else {
            throw out_of_range("Nomor produk tidak ditemukan.");
        }
    } catch (const exception& e) {
        cout << RED << "\n[ERROR] Terjadi Kesalahan: " << e.what() << RESET <<
endl;
    }
}

void hapusPesanan(Pesanan dataPesanan[], int &jumlahPesanan) {
    try {
        string tempHapus; cout << "\nNomor pesanan: "; getline(cin, tempHapus);
        if (tempHapus.empty()) throw invalid_argument("Nomor pesanan tidak boleh
kosong.");
        if (!all_of(tempHapus.begin(), tempHapus.end(), ::isdigit)) throw
invalid_argument("Nomor pesanan harus berupa angka.");
```

```
        int hapus = stoi(tempHapus);

        if (hapus > 0 && hapus <= jumlahPesanan) {
            for (int i = hapus-1; i < jumlahPesanan-1; i++) {
                dataPesanan[i] = dataPesanan[i+1];
            }
            jumlahPesanan--;
            cout << GREEN << "\n[SUCCESS] Pesanan berhasil dihapus.\n" << RESET;
        } else {
            throw out_of_range("Nomor pesanan tidak ditemukan.");
        }
    } catch (const exception& e) {
        cout << RED << "\n[ERROR] Terjadi Kesalahan: " << e.what() << RESET <<
endl;
    }
}

int bayarPesanan(Pesanan dataPesanan[], Produk dataProduk[], int n, bool
rekursif) {
    if (n <= 0) return 0;
    int totalSebelumnya = bayarPesanan(dataPesanan, dataProduk, n - 1, true);

    int idx = dataPesanan[n - 1].idxProduk;
    int harga = dataProduk[idx].detail.harga;
    int jumlah = dataPesanan[n - 1].jumlah;
    int subtotal = hitungTotalDasar(harga, jumlah);

    cout << RESET << left << setw(4) << n << setw(32) << dataProduk[idx].name <<
"Rp" << setw(10) << harga << setw(10) << jumlah << "Rp" << subtotal << endl;

    return totalSebelumnya + subtotal;
}

void bayarPesanan(Pesanan dataPesanan[], Produk dataProduk[], int
&jumlahPesanan) {
    if (jumlahPesanan == 0) {
        cout << YELLOW << "Belum ada pesanan." << RESET;
    } else {
        try {
            cout << CYAN <<
"=========================================================================\n";
            cout << left << setw(4) << "No" << setw(32) << "Nama Produk" <<
setw(12) << "Harga" << setw(10) << "Jumlah" << "Subtotal\n";
            cout <<
"=========================================================================\n";
            int totalSebelumPajak = bayarPesanan(dataPesanan, dataProduk,
jumlahPesanan, true);

            cout <<
"=========================================================================\n";
```

```
            int totalDenganPajak = hitungDenganPajak(totalSebelumPajak, 5);

            int totalAkhir = totalDenganPajak;
            if (totalSebelumPajak > 100000) {
                totalAkhir = hitungDenganDiskon(totalDenganPajak, 10);
                cout << RESET << "Total        : Rp" << totalSebelumPajak <<
endl;
                cout << "Total + Pajak (5%): Rp" << totalDenganPajak << endl;
                cout << GREEN << "Total - Diskon 10%: Rp" << totalAkhir << RESET
<< endl;
            } else {
                cout << RESET << "Total        : Rp" << totalSebelumPajak <<
endl;
                cout << GREEN << "Total + Pajak (5%): Rp" << totalDenganPajak <<
RESET << endl;
            }
            string konfirmasi; cout << "\nKonfirmasi pembayaran (y/n): ";
getline(cin, konfirmasi);
            if (konfirmasi.empty()) throw invalid_argument("Input tidak boleh
kosong!");

            if (konfirmasi == "y" || konfirmasi == "Y") {
                for (int i = 0; i < jumlahPesanan; i++) {
                    int idx = dataPesanan[i].idxProduk;
                    dataProduk[idx].detail.stok -= dataPesanan[i].jumlah;
                }
                jumlahPesanan = 0;
                cout << "[SUCCESS] Pembayaran berhasil sebesar Rp" << totalAkhir
<< ". Terima kasih telah berbelanja!\n";
            } else if (konfirmasi == "n" || konfirmasi == "N") {
                cout << YELLOW << "[FAILED] Pembayaran dibatalkan.\n" << RESET;
                return;
            } else {
                throw invalid_argument("Pilihan konfirmasi tidak valid. Gunakan
'y' atau 'n'.");
            }
        } catch (const exception& e) {
            cout << RED << "\n[ERROR] Gagal memproses pembayaran: " << e.what()
<< RESET << endl;
        }
    }
}
```

I. Fitur Fungsi Rekursif dan Overloading

Pada program ini terdapat dua fungsi dengan nama yang sama yaitu bayarPesanan, yang menerapkan konsep function overloading. Kedua fungsi tersebut memiliki parameter yang berbeda dan fungsi yang saling melengkapi dalam proses pembayaran.

Source Code:

```
int bayarPesanan(Pesanan dataPesanan[], Produk dataProduk[], int n, bool
rekursif) {
    if (n <= 0) return 0;
    int totalSebelumnya = bayarPesanan(dataPesanan, dataProduk, n - 1, true);

    int idx = dataPesanan[n - 1].idxProduk;
    int harga = dataProduk[idx].detail.harga;
    int jumlah = dataPesanan[n - 1].jumlah;
    int subtotal = hitungTotalDasar(harga, jumlah);

    cout << RESET << left << setw(4) << n << setw(32) << dataProduk[idx].name <<
"Rp" << setw(10) << harga << setw(10) << jumlah << "Rp" << subtotal << endl;

    return totalSebelumnya + subtotal;
}

void bayarPesanan(Pesanan dataPesanan[], Produk dataProduk[], int
&jumlahPesanan) {
    if (jumlahPesanan == 0) {
        cout << YELLOW << "Belum ada pesanan." << RESET;
    } else {
        try {
            cout << CYAN <<
"=========================================================================\n";
            cout << left << setw(4) << "No" << setw(32) << "Nama Produk" <<
setw(12) << "Harga" << setw(10) << "Jumlah" << "Subtotal\n";
            cout <<
"=========================================================================\n";
            int totalSebelumPajak = bayarPesanan(dataPesanan, dataProduk,
jumlahPesanan, true);

            cout <<
"=========================================================================\n";

            int totalDenganPajak = hitungDenganPajak(totalSebelumPajak, 5);

            int totalAkhir = totalDenganPajak;
            if (totalSebelumPajak > 100000) {
                totalAkhir = hitungDenganDiskon(totalDenganPajak, 10);
                cout << RESET << "Total        : Rp" << totalSebelumPajak <<
```

```
endl;
                cout << "Total + Pajak (5%): Rp" << totalDenganPajak << endl;
                cout << GREEN << "Total - Diskon 10%: Rp" << totalAkhir << RESET
<< endl;
            } else {
                cout << RESET << "Total        : Rp" << totalSebelumPajak <<
endl;
                cout << GREEN << "Total + Pajak (5%): Rp" << totalDenganPajak <<
RESET << endl;
            }
            string konfirmasi; cout << "\nKonfirmasi pembayaran (y/n): ";
getline(cin, konfirmasi);
            if (konfirmasi.empty()) throw invalid_argument("Input tidak boleh
kosong!");

            if (konfirmasi == "y" || konfirmasi == "Y") {
                for (int i = 0; i < jumlahPesanan; i++) {
                    int idx = dataPesanan[i].idxProduk;
                    dataProduk[idx].detail.stok -= dataPesanan[i].jumlah;
                }
                jumlahPesanan = 0;
                cout << "[SUCCESS] Pembayaran berhasil sebesar Rp" << totalAkhir
<< ". Terima kasih telah berbelanja!\n";
            } else if (konfirmasi == "n" || konfirmasi == "N") {
                cout << YELLOW << "[FAILED] Pembayaran dibatalkan.\n" << RESET;
                return;
            } else {
                throw invalid_argument("Pilihan konfirmasi tidak valid. Gunakan
'y' atau 'n'.");
            }
        } catch (const exception& e) {
            cout << RED << "\n[ERROR] Gagal memproses pembayaran: " << e.what()
<< RESET << endl;
        }
    }
}
```

J. Fitur Membersihkan Layar

Sistem menyediakan fungsi clearScreen untuk membersihkan layar sesuai sistem operasi yang digunakan.

Source Code:

```
void clearScreen() {
#if defined _WIN32
```

```
    system("cls");
#elif defined (__LINUX__) || defined(__gnu_linux__) || defined(__linux__)
    system("clear");
#elif defined (__APPLE__)
    system("clear");
#endif
//Kode untuk membersihkan layar berdasarkan sistem operasi yang digunakan ini
saya dapatkan dari website Stack Overflow.
}
```

K. Fitur Pause

Prosedur ini digunakan sebagai fitur untuk memberi jeda sebelum pengguna menekan enter untuk ke menu selanjutnya yang akan ditampilkan program.

Source Code:

```
void pause() {
    cout << YELLOW << "\nTekan Enter untuk melanjutkan..." << RESET;
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}
```

L. Fitur Perhitungan Toko

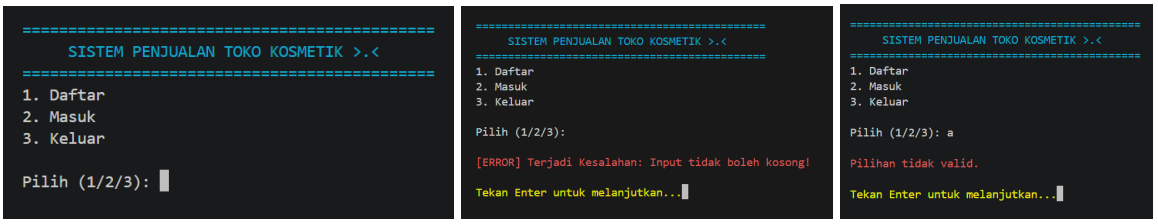
Library eksternal hitung_toko.h digunakan sebagai modul pendukung dalam proses perhitungan pada sistem pembayaran di menu pembeli. Library ini berada dalam namespace TokoLib dan berisi beberapa fungsi penting, yaitu hitungTotalDasar() untuk menghitung subtotal harga, hitungDenganPajak() untuk menambahkan pajak, serta hitungDenganDiskon() untuk mengurangi total dengan diskon. Selain itu, terdapat fungsi validasiHarga() yang digunakan untuk memastikan harga produk sesuai dengan ketentuan minimal.

Source Code:

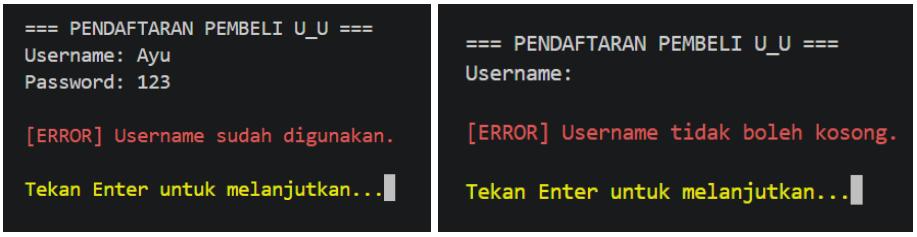
```
#ifndef HITUNG_TOKO_H
#define HITUNG_TOKO_H
#include <stdexcept>

namespace TokoLib {
```

```
    inline int hitungTotalDasar(int harga, int jumlah) {
        if (jumlah < 0 || harga < 0) {
            throw std::invalid_argument("Jumlah tidak boleh negatif!");
        }
        return harga * jumlah;
    }

    inline int hitungDenganPajak(int total, int persenPajak) {
        if (persenPajak < 0 || persenPajak > 100) throw
std::invalid_argument("Pajak tidak valid (0-100)!");
        return total + (total * persenPajak / 100);
    }

    inline int hitungDenganDiskon(int total, int persenDiskon) {
        if (persenDiskon < 0 || persenDiskon > 100) {
            throw std::invalid_argument("Diskon tidak valid (0-100)!");
        }
        return total - (total * persenDiskon / 100);
    }

    inline bool validasiHarga(int harga) {
        return harga >= 20000;
    }
}

#endif
```

4. Hasil Output



Gambar 4.1 Menu Utama



Gambar 4.2 Fitur Daftar Gagal

```
=== PENDAFTARAN PEMBELI U_U ===
Username: Ayu1
Password: 123
Pendaftaran berhasil.

Tekan Enter untuk melanjutkan...
```

Gambar 4.3 Fitur Daftar Berhasil

```
=== MASUK KE AKUN O,O ===
Username: Aya
Password: 062

Selamat! Anda berhasil masuk sebagai ADMIN.

Tekan Enter untuk melanjutkan...
```

Gambar 4.4 Fitur Masuk Admin Berhasil

```
=== MASUK KE AKUN O,O ===
Username: Ayu
Password: 213

[ERROR] Username atau password salah!

Sisa kesempatan: 1
Silakan coba lagi.

Tekan Enter untuk melanjutkan...
```

Gambar 4.5 Fitur Masuk Gagal

```
=== MASUK KE AKUN O,O ===
Username:

[ERROR] Username tidak boleh kosong.

Sisa kesempatan: 0
Kesempatan login habis! Silakan coba beberapa saat lagi.
```

Gambar 4.6 Fitur Masuk Gagal Setelah 3 Kesempatan Habis

```
=== MENU ADMIN >u< ===
1. Tambah Produk
2. Lihat Produk
3. Ubah Produk
4. Hapus Produk
5. Urutkan Produk
6. Cari Produk
7. Keluar
---------------------
Pilih:
```

Gambar 4.7 Menu Admin

```
=== TAMBAH PRODUK (z'v'z) ===
Nama Produk: AYE! Lip Serum
Kategori: Lip Makeup
Harga: a

[ERROR] Terjadi Kesalahan: Harga tidak valid.

Tekan Enter untuk melanjutkan...
```

```
=== TAMBAH PRODUK (z'v'z) ===
Nama Produk: AYE! Lip Serum
Kategori: Lip Makeup
Harga:

[ERROR] Terjadi Kesalahan: Harga tidak boleh kosong.

Tekan Enter untuk melanjutkan...
```

Gambar 4.8 Fitur Tambah Produk Gagal 1

```
=== TAMBAH PRODUK (z'v'z) ===
Nama Produk: AYE! Tinted Balm
Kategori: Lip Makeup
Harga: 25000
Stok: 10

[SUCCESS] Produk berhasil ditambahkan.

Tekan Enter untuk melanjutkan...
```

Gambar 4.9 Fitur Tambah Produk Berhasil

```
=== TAMBAH PRODUK (z'v'z) ===
Nama Produk: AYE! Lip Serum
Kategori: Lip Makeup
Harga: 27000
Stok: a

[ERROR] Terjadi Kesalahan: Stok tidak valid.

Tekan Enter untuk melanjutkan...
```

```
=== TAMBAH PRODUK (z'v'z) ===
Nama Produk: AYE! Lip Serum
Kategori: Lip Makeup
Harga: 27000
Stok:

[ERROR] Terjadi Kesalahan: Stok tidak boleh kosong.

Tekan Enter untuk melanjutkan...
```

Gambar 4.10 Fitur Tambah Produk Gagal 2

```
=====================================================================
No  Nama Produk                    Kategori        Harga       Stok
=====================================================================
1   AYE! Nude Eyeshadow Pallete    Eye Makeup      Rp55000     10
2   AYE! Coral Glossy Tint         Lip Makeup      Rp35000     10
3   AYE! Autumn Brown Blush        Cheek Makeup    Rp40000     10
4   AYE! Curling Black Mascara     Eye Makeup      Rp79000     10
5   AYE! Ivory Cushion W/ SPF++    Base Makeup     Rp98000     10
6   AYE! Tinted Balm               Lip Makeup      Rp25000     10

Tekan Enter untuk melanjutkan...
```

Gambar 4.11 Fitur Lihat produk Admin & Pembeli

```
Nomor produk:

[ERROR] Terjadi Kesalahan: Nomor produk tidak boleh kosong.

Tekan Enter untuk melanjutkan...
```

Gambar 4.12 Fitur Ubah/Hapus Produk Gagal

```
Nomor produk: 1
Harga baru: 56000
Stok baru: 10

[SUCCESS] Produk berhasil diperbarui.

Tekan Enter untuk melanjutkan...
```

Gambar 4.13 Fitur Ubah Produk Berhasil

```
==========================================================================
No  Nama Produk                    Kategori          Harga       Stok
==========================================================================
1   AYE! Nude Eyeshadow Pallete    Eye Makeup        Rp56000     10
2   AYE! Coral Glossy Tint         Lip Makeup        Rp35000     10
3   AYE! Autumn Brown Blush        Cheek Makeup      Rp40000     10
4   AYE! Curling Black Mascara     Eye Makeup        Rp79000     10
5   AYE! Ivory Cushion W/ SPF++    Base Makeup       Rp98000     10
6   AYE! Tinted Balm               Lip Makeup        Rp25000     10

Nomor produk: 6

[SUCCESS] Produk berhasil dihapus.

Tekan Enter untuk melanjutkan...
```

Gambar 4.14 Fitur Hapus Produk

```
Terima kasih telah mengunjungi AYE'S COSMETICS.
PS C:\Users\ASUS\Documents\praktikum-apl\post-test\post-test-apl-7>
```

Gambar 4.15 Fitur Keluar

```
=== MASUK KE AKUN O,O ===
Username: Ayu
Password: 123

Selamat! Anda berhasil masuk sebagai PEMBELI.

Tekan Enter untuk melanjutkan...
```

Gambar 4.16 Fitur Masuk Pembeli

```
==== MENU PEMBELI ^w^ ====
1. Buat Pesanan
2. Lihat Produk & Pesanan
3. Hapus Pesanan
4. Bayar Pesanan
5. Keluar
--------------------------
Pilih:
```

Gambar 4.17 Menu Pembeli

```
==========================================================================
No  Nama Produk                    Kategori          Harga       Stok
==========================================================================
1   AYE! Nude Eyeshadow Pallete    Eye Makeup        Rp55000     10
2   AYE! Coral Glossy Tint         Lip Makeup        Rp35000     10
3   AYE! Autumn Brown Blush        Cheek Makeup      Rp40000     10
4   AYE! Curling Black Mascara     Eye Makeup        Rp79000     10
5   AYE! Ivory Cushion W/ SPF++    Base Makeup       Rp98000     10

Pilih nomor produk: 1
Jumlah beli: 1

[SUCCESS] Pesanan berhasil ditambahkan.

Tekan Enter untuk melanjutkan...
```

Gambar 4.18 Fitur Buat Pesanan

```
==========================================================================
No  Nama Produk                    Kategori         Harga       Stok
==========================================================================
1   AYE! Nude Eyeshadow Pallete    Eye Makeup       Rp55000     10
2   AYE! Coral Glossy Tint         Lip Makeup       Rp35000     10
3   AYE! Autumn Brown Blush        Cheek Makeup     Rp40000     10
4   AYE! Curling Black Mascara     Eye Makeup       Rp79000     10
5   AYE! Ivory Cushion W/ SPF++    Base Makeup      Rp98000     10

==== PESANAN ANDA ====
1. AYE! Nude Eyeshadow Pallete | Jumlah: 1

Tekan Enter untuk melanjutkan...
```

Gambar 4.19 Fitur Lihat Produk & Pesanan

```
==== PESANAN ANDA ====
1. AYE! Nude Eyeshadow Pallete | Jumlah: 1

Nomor pesanan: 1

[SUCCESS] Pesanan berhasil dihapus.

Tekan Enter untuk melanjutkan...
```

Gambar 4.20 Fitur Hapus Pesanan Berhasil

```
Nomor pesanan:

[ERROR] Terjadi Kesalahan: Nomor pesanan tidak boleh kosong.

Tekan Enter untuk melanjutkan...
```

Gambar 4.21 Fitur Hapus Pesanan Gagal

```
==========================================================================
No  Nama Produk                    Harga       Jumlah    Subtotal
==========================================================================
1   AYE! Nude Eyeshadow Pallete    Rp55000     1         Rp55000
==========================================================================
Total        : Rp55000
Total + Pajak (5%): Rp57750

Konfirmasi pembayaran (y/n): y
[SUCCESS] Pembayaran berhasil sebesar Rp57750. Terima kasih telah berbelanja!

Tekan Enter untuk melanjutkan...
```

Gambar 4.22 Fitur Bayar Pesanan Berhasil

```
==========================================================================
No  Nama Produk                    Harga       Jumlah    Subtotal
==========================================================================
1   AYE! Nude Eyeshadow Pallete    Rp55000     1         Rp55000
==========================================================================
Total        : Rp55000
Total + Pajak (5%): Rp57750

Konfirmasi pembayaran (y/n): n
[FAILED] Pembayaran dibatalkan.

Tekan Enter untuk melanjutkan...
```

```
==========================================================================
No  Nama Produk                    Harga       Jumlah    Subtotal
==========================================================================
1   AYE! Nude Eyeshadow Pallete    Rp55000     1         Rp55000
==========================================================================
Total        : Rp55000
Total + Pajak (5%): Rp57750

Konfirmasi pembayaran (y/n): m

[ERROR] Gagal memproses pembayaran: Pilihan konfirmasi tidak valid. Gunakan 'y' atau 'n'.

Tekan Enter untuk melanjutkan...
```

Gambar 4.23 Fitur Bayar Pesanan Batal

```
Belum ada pesanan.
Tekan Enter untuk melanjutkan...
```

```
==== PESANAN ANDA ====
Belum ada pesanan.

Tekan Enter untuk melanjutkan...
```

Gambar 4.24 Fitur Bayar & Lihat Pesanan Kosong

```
======== MENU PENGURUTAN PRODUK >;< ========
1. Sort Berdasarkan Nama (Z-A)
2. Sort Berdasarkan Harga (Murah-Mahal)
3. Sort Berdasarkan Stok (Terbanyak)
-----------------------------------------
Pilih: 1
Produk berhasil diurutkan (Nama Z-A).
=========================================================================
No  Nama Produk                    Kategori          Harga       Stok
=========================================================================
1   AYE! Nude Eyeshadow Pallete    Eye Makeup        Rp55000     9
2   AYE! Ivory Cushion W/ SPF++    Base Makeup       Rp98000     10
3   AYE! Curling Black Mascara     Eye Makeup        Rp79000     10
4   AYE! Coral Glossy Tint         Lip Makeup        Rp35000     10
5   AYE! Autumn Brown Blush        Cheek Makeup      Rp40000     10

Tekan Enter untuk melanjutkan...
```

Gambar 4.25 Fitur Urutkan Produk di Menu Admin

```
===== MENU SEARCHING =====
1. Cari berdasarkan Harga
2. Cari berdasarkan Nama
-------------------------
Pilih: 1

Masukkan harga yang dicari: 55000

Produk ditemukan:
AYE! Nude Eyeshadow Pallete | Eye Makeup | Rp55000

Tekan Enter untuk melanjutkan...
```

Gambar 4.26 Fitur Cari Produk di Menu Admin

```
======== MENU PENGURUTAN PRODUK >;< ========
1. Sort Berdasarkan Nama (Z-A)
2. Sort Berdasarkan Harga (Murah-Mahal)
3. Sort Berdasarkan Stok (Terbanyak)
-----------------------------------------
Pilih:

[ERROR] Input tidak boleh kosong!

Tekan Enter untuk melanjutkan...
```

```
===== MENU SEARCHING =====
1. Cari berdasarkan Harga
2. Cari berdasarkan Nama
-------------------------
Pilih: a
Pilihan tidak valid.

Tekan Enter untuk melanjutkan...
```

Gambar 4.27 Pesan Error Urutkan/Cari Produk di Menu Admin

5. Langkah-langkah GIT

5.1 GIT Init

```
PS C:\Users\ASUS\Documents\praktikum-apl\post-test\post-test-apl-2> git init
Initialized empty Git repository in C:/Users/ASUS/Documents/praktikum-apl/post-test/post-test-apl-2/.git/
PS C:\Users\ASUS\Documents\praktikum-apl\post-test\post-test-apl-2>
```

Git init berfungsi untuk menginisialisasi repository lokal.

5.2 GIT Add

```
PS C:\Users\ASUS\Documents\praktikum-apl> git add .
PS C:\Users\ASUS\Documents\praktikum-apl>
```

Git add . berfungsi untuk menambahkan semua file baru pada folder praktikum-apd di repository.

5.3 GIT Commit

```
PS C:\Users\ASUS\Documents\praktikum-apl> git commit -m "Posting Post Test 7 APL"
[main 0f94d36] Posting Post Test 7 APL
 3 files changed, 697 insertions(+)
 create mode 100644 post-test/post-test-apl-7/2509106062-INTANAYURAHMAPUTRI-PT-7.cpp
 create mode 100644 post-test/post-test-apl-7/2509106062-INTANAYURAHMAPUTRI-PT-7.exe
 create mode 100644 post-test/post-test-apl-7/hitung_toko.h
```

Git commit berfungsi untuk memberikan pesan “Posting Post Test 7 APL” sebagai konfirmasi perubahan yang terjadi pada repository praktikum-apl.

5.4 GIT Remote

```
PS C:\Users\ASUS\Documents\praktikum-apl\post-test\post-test-apl-2> git remote add origin https://github.com/intanayurahmap
utri/praktikum-apl.git
```

Git Remote berfungsi untuk menghubungkan file yang ada di repository lokal ke repository Github.

5.5 GIT Push

```
PS C:\Users\ASUS\Documents\praktikum-apl> git push -u origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 8 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 41.86 KiB | 1.31 MiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/intanayurahmaputri/praktikum-apl.git
   ccd61b7..0f94d36  main -> main
branch 'main' set up to track 'origin/main'.
```

Git push berfungsi untuk memposting file yang tadinya hanya berada pada komputer ke Github.